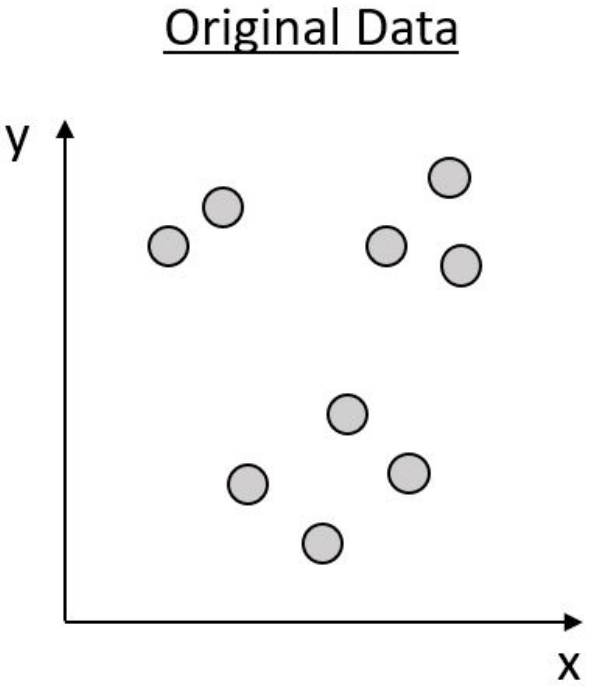
Clustering

Hrachya Asatryan

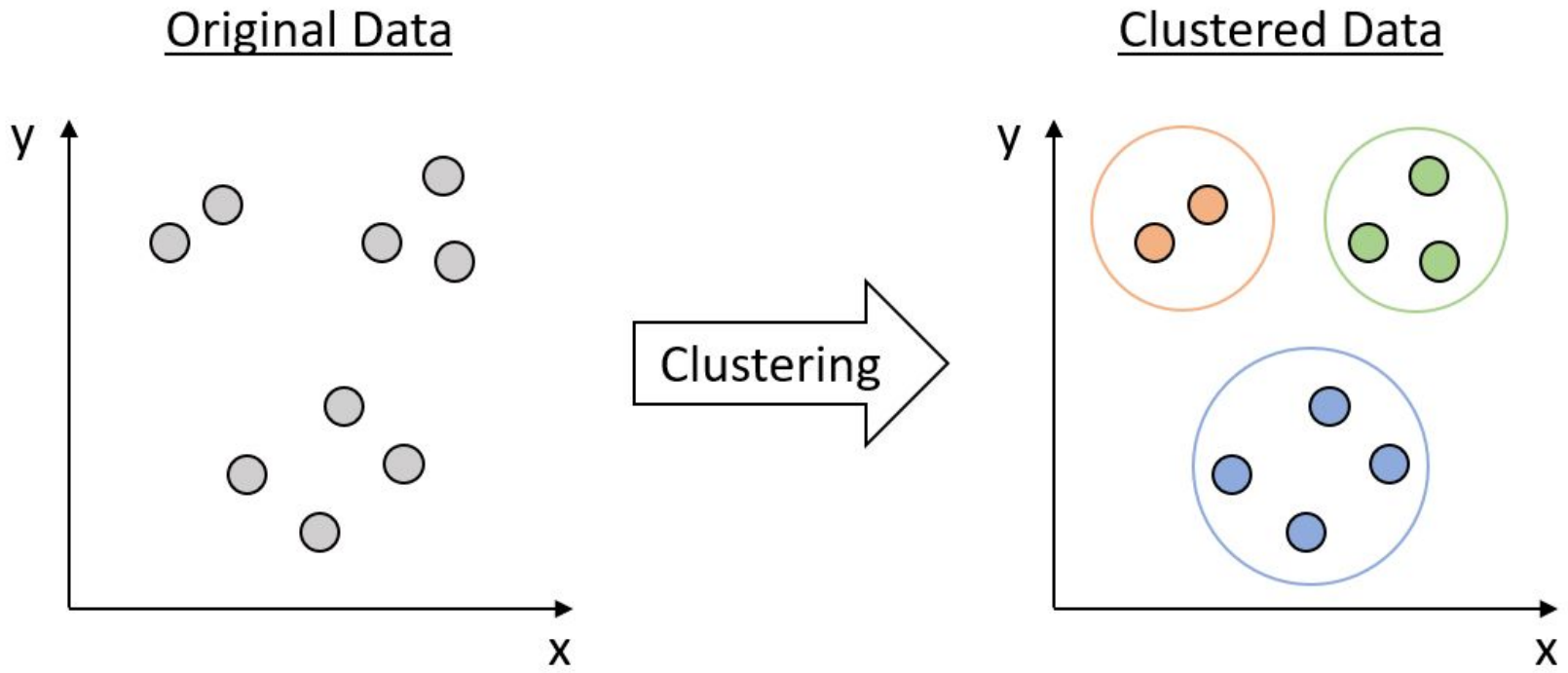
What is clustering?



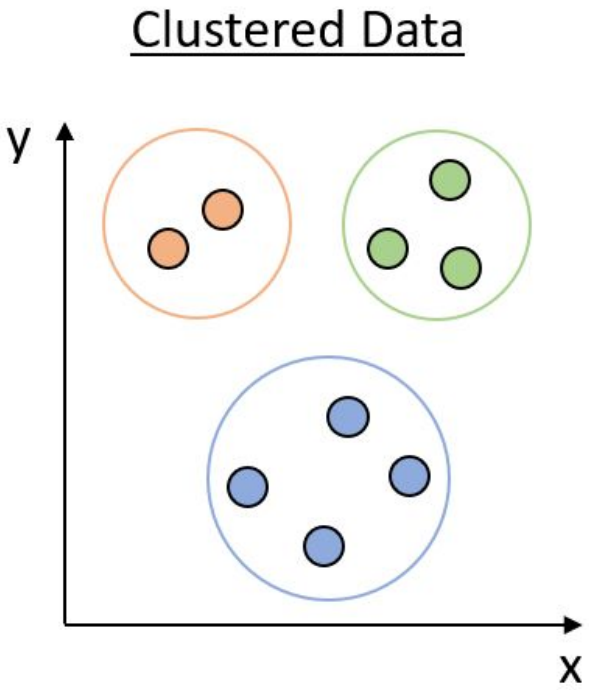
We are given unlabelled data objects.

Our objective is to find a grouping for said objects, thus predict labels.

What is clustering?

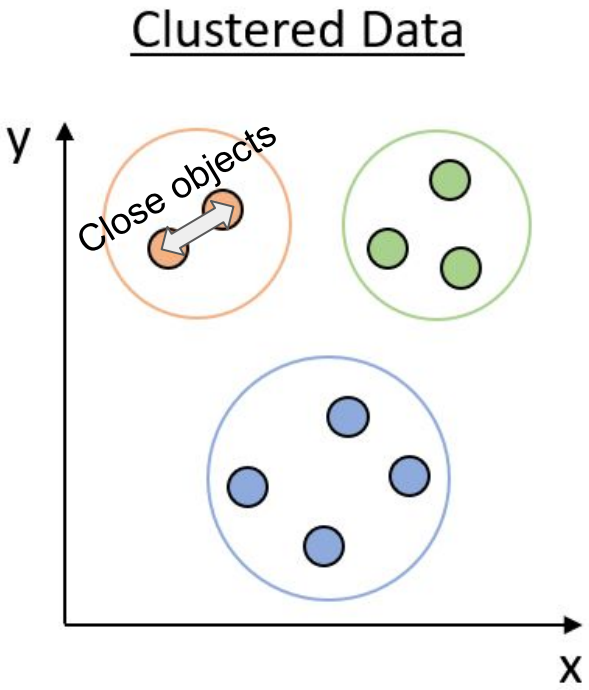


What is clustering?



We want to find the optimal grouping. Thus we come up with some rules to differentiate good and bad clusters.

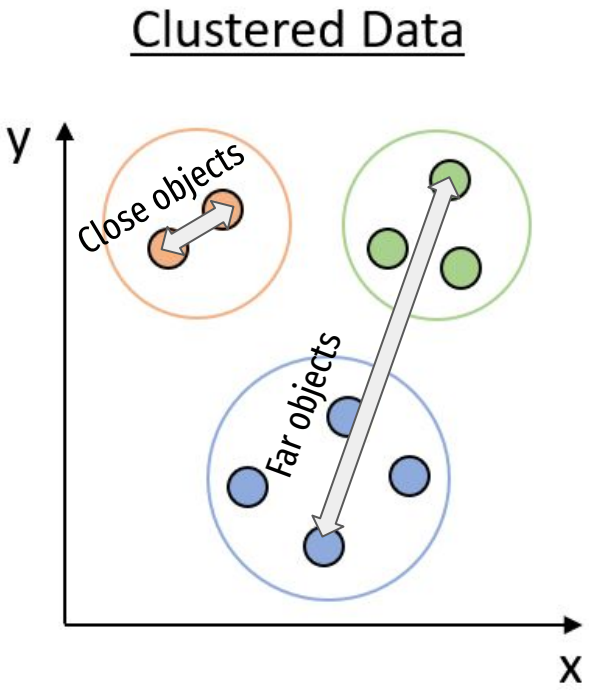
What is clustering?



We want to find the optimal grouping. Thus we come up with some rules to differentiate good and bad clusters.

- We want to have close objects in the same group

What is clustering?



We want to find the optimal grouping. Thus we come up with some rules to differentiate good and bad clusters.

- We want to have close objects in the same group
- And far apart objects in different groups

Why do we cluster data?

- Discover relationships between the observations
- Intuition building
- Hypothesis generation
- Discover structures and patterns in high-dimensional data
- Summarizing / compressing large data

Clustering is subjective

Suppose that we need to put you in some groups for projects.

How to define those groups?

- Work experience
- Age
- Education
- Preferences

Which way is correct? Depends on the goal!

Examples

- Anomaly detection: Clustering is used to detect anomalies in data, such as fraudulent financial transactions, unusual patterns in network traffic, or outliers in medical data.

Examples

- Social Network Analysis: Clustering is used on social network users to group individuals with similar social connections and characteristics together, allowing for the identification of subgroups within a larger network.

Examples

- Biology and Medicine: Cluster analysis is used in biology and medicine to identify genes associated with specific diseases or to group patients with similar clinical characteristics together. This can help with the diagnosis and treatment of diseases.

Examples

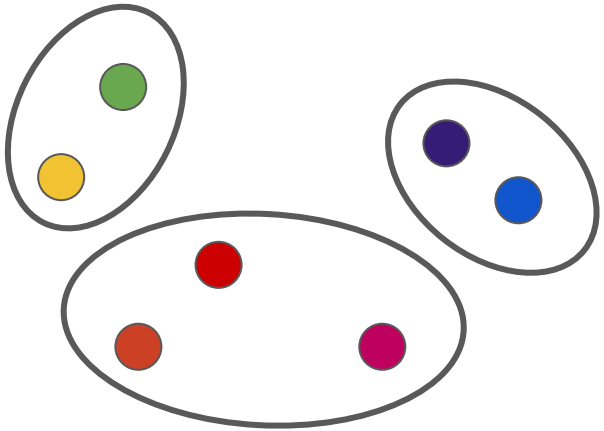
- Image Processing: In image processing, cluster analysis is used to group pixels with similar properties together, allowing for the identification of objects and patterns in images. Alternatively, the main colors of the image can be determined and used for recoloring.

Examples

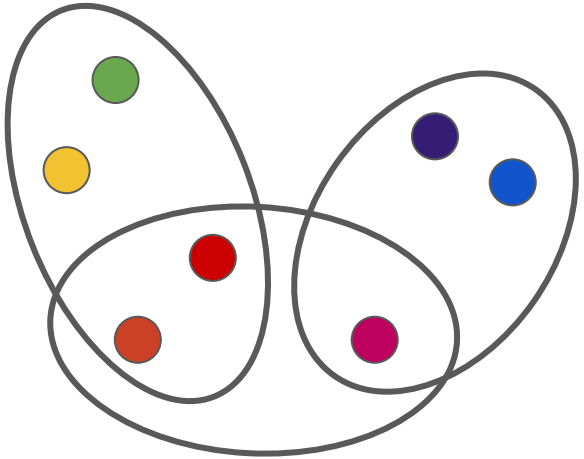
- Market Segmentation: Cluster analysis is often used in marketing to segment customers into groups based on their buying behavior, demographics, or other characteristics. This information can be used to create targeted marketing campaigns or to develop new products that appeal to specific customer groups.

Clustering types

- **Fuzzy (Soft) clustering**: Each object belongs to each cluster with some probability/weight (the weight can be zero).
- **Non-fuzzy (Hard) clustering**: each object belongs to exactly one cluster.



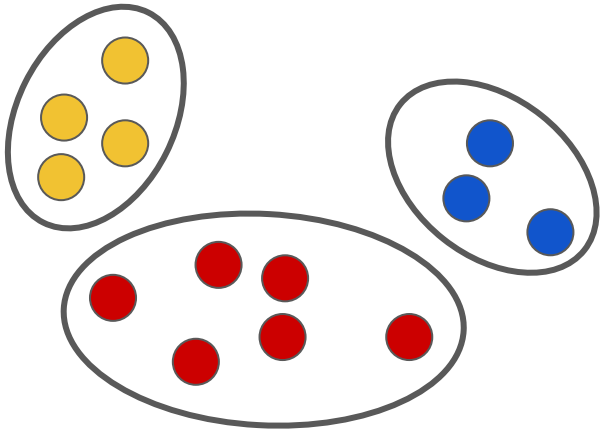
Hard



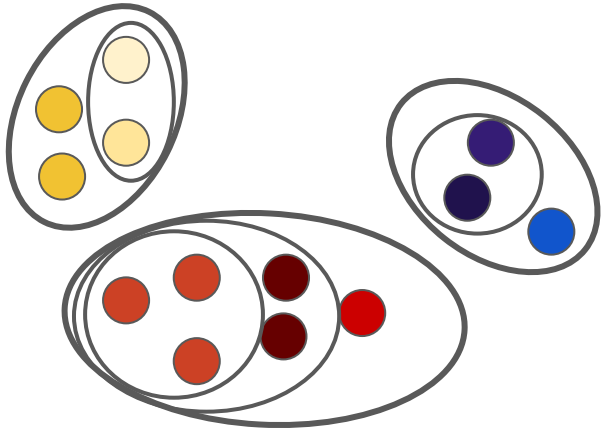
Soft

Hierarchical clustering

- **Partitional clustering**: finds a fixed number of clusters
- **Hierarchical clustering**: creates a series of clusterings contained in each other (nested clusters)



Partitional

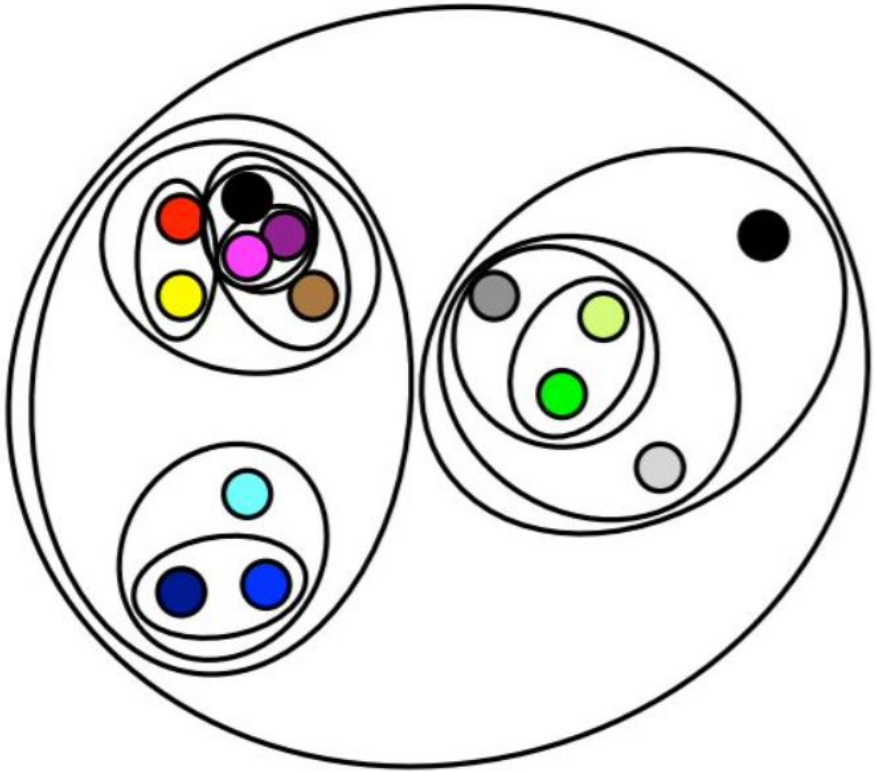


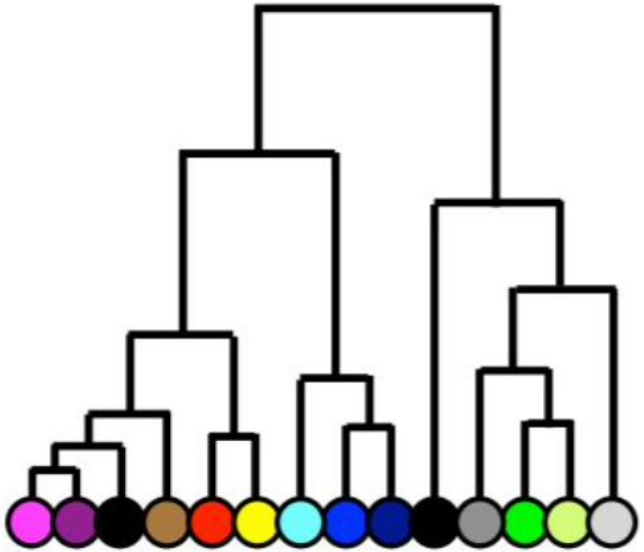
Hierarchical

Hierarchical clustering

- Hierarchical clustering is usually visualized as a dendrogram (tree)
- Each subtree corresponds to a cluster
- Height of branching shows distance between the objects
- There are two strategies: Agglomerative and Divisive
  - **Agglomerative clustering (Bottom-up):** Start with a single-object clusters (singletons) and recursively merge them into larger clusters.
  - **Divisive clustering (Top down):** Start with a cluster containing all data points and recursively divide it into smaller clusters.

Hierarchical clustering

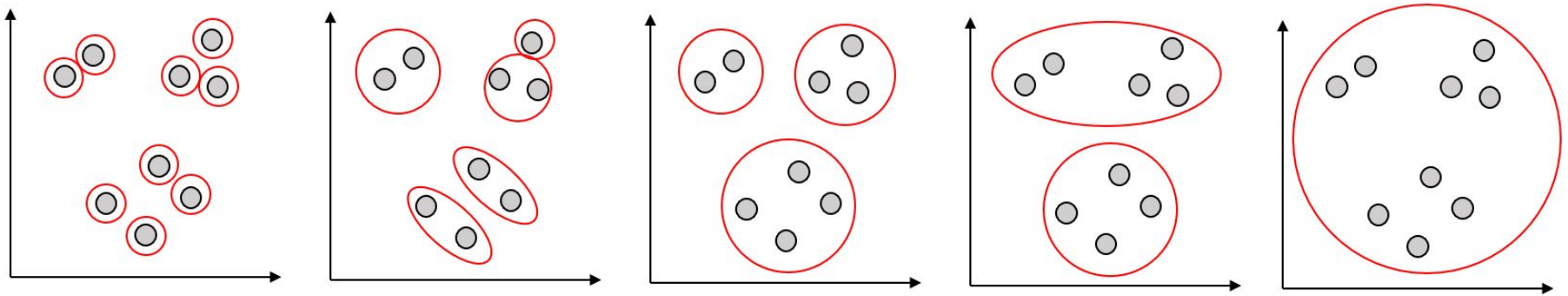




Hierarchical clustering: Bottom up algorithm

Assumption: Each object is its own cluster

1. Identify two most similar clusters (based on some metric, let's say Euclidean distance)
2. Merge the two clusters
3. Repeat!



Hierarchical clustering: Linkage

Specifying how clusters are assigned will influence the final shape and contents of the clusters. **Linkage** criteria determines how the distance between clusters is measured, and consequently which clusters will be group during the next iteration. Three common linkage criteria are:

- **Complete (max):** distance is computed between the two **least** similar parts of clusters (two most distant points)
- **Single (min):** distance is computed between the two **most** similar parts of clusters (two closest points)
- **Average:** distance is computed between clusters' centroids

Hierarchical clustering: Linkage

- **Complete (max):** `max{d(a, b) : a in A, b in B}`
  - Suffers from chaining meaning that clusters can be too spread out, and not compact enough
  - Clusters can violate the “compactness” property that all observations within each cluster tend to be similar to one another.
- **Single (min):** `min{d(a, b) : a in A, b in B}`
  - Avoids chaining, but suffers from crowding, meaning that clusters are compact, but not far enough apart
  - Observations assigned to a cluster can be much closer to members of other clusters than they are to some members of their own cluster
- **Average:** `average{ d(a, b) : a in A, b in B}`
  - Balanced approach: clusters tend to be relatively compact and relatively far apart.

Hierarchical clustering: Divisive methods

- Divisive methods start at the top and at each level recursively split one of the existing clusters at that level into two new clusters.
- The split is chosen to produce two new groups with the largest between-group dissimilarity.
- Recursively divide one of the existing clusters into two daughter clusters at each iteration in a top-down fashion.

Hierarchical clustering: Divisive methods

- This approach has not been studied nearly as extensively as agglomerative methods in the clustering literature.
- It has been explored somewhat in the engineering literature (Gersho and Gray, 1992) in the context of compression.
- In the clustering setting, a potential advantage of divisive over agglomerative methods can occur when interest is focused on partitioning the data into a relatively small number of clusters.
- The divisive paradigm can be employed by recursively applying any of the combinatorial methods such as K-means or K-medoids, with K = 2, to perform the splits at each iteration.
- However, such an approach would depend on the starting configuration specified at each step.

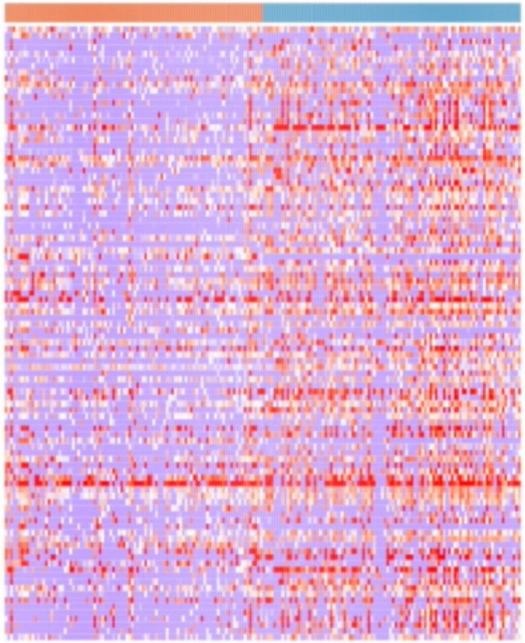
Hierarchical clustering: Divisive methods

- A divisive algorithm that avoids these problems was proposed by Macnaughton Smith et al. (1965).
- It begins by placing all observations in a single cluster G.
- It then chooses that observation whose average dissimilarity from all the other observations is largest.
- This observation forms the first member of a second cluster H.
- At each successive step that observation in G whose average distance from those in H, minus that for the remaining observations in G is largest, is transferred to H.
- This continues until the corresponding difference in averages becomes negative. That is, there are no longer any observations in G that are, on average, closer to those in H.

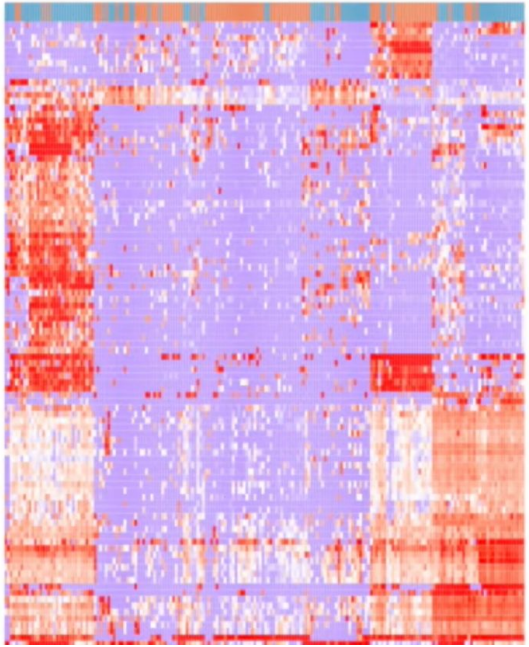
Hierarchical clustering: Pros and Cons

- **Pro**: Hierarchical clustering works well with histograms, making the data more sensible!

Without hierarchical clustering...

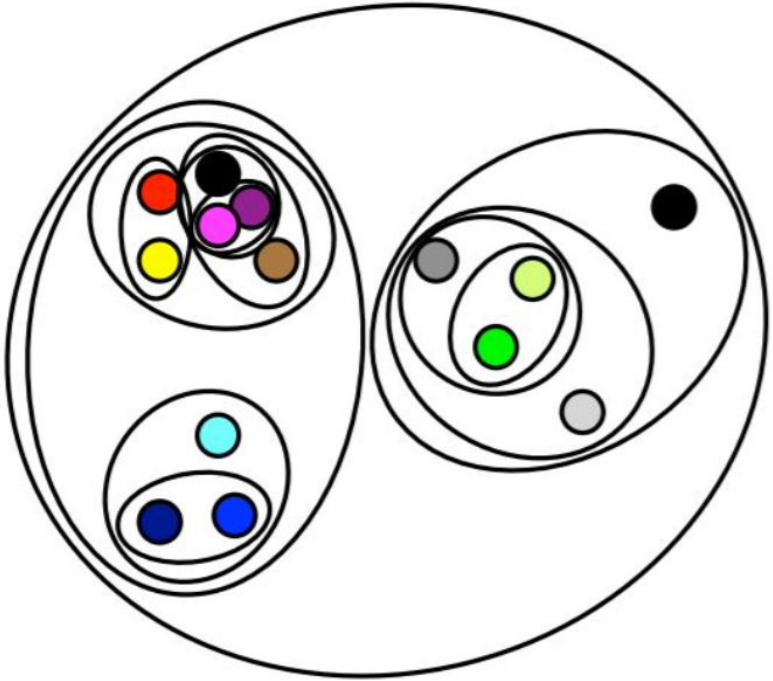


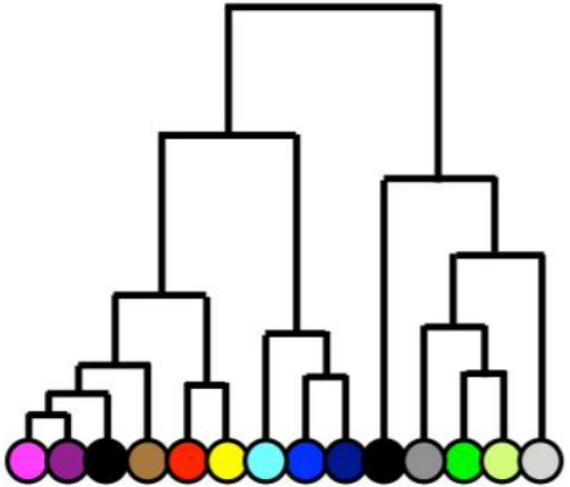
...with hierarchical clustering



Hierarchical clustering: Pros and Cons

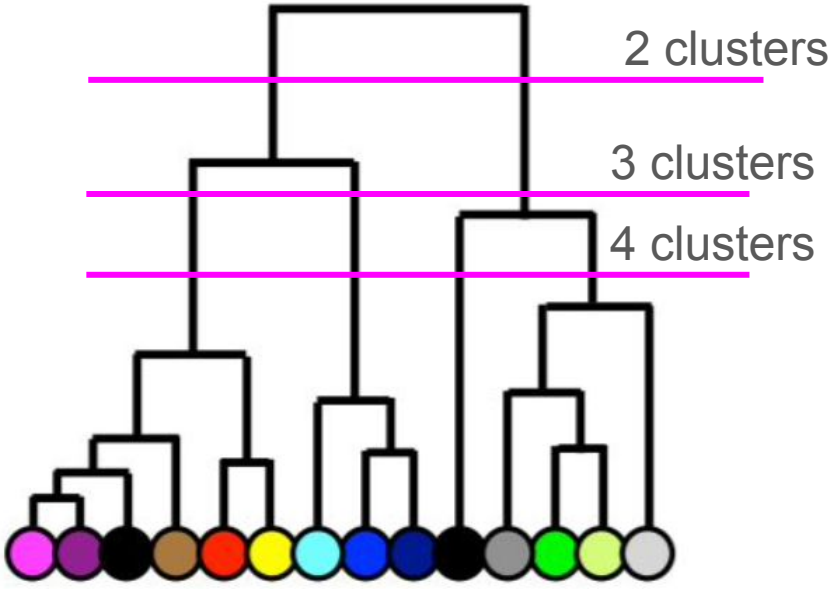
- **Pro**: The clusters can be visualized and interpreted with dendrograms





Hierarchical clustering: Pros and Cons

- **Pro**: The number of clusters is not predefined and can be chosen after inspecting the dendrogram

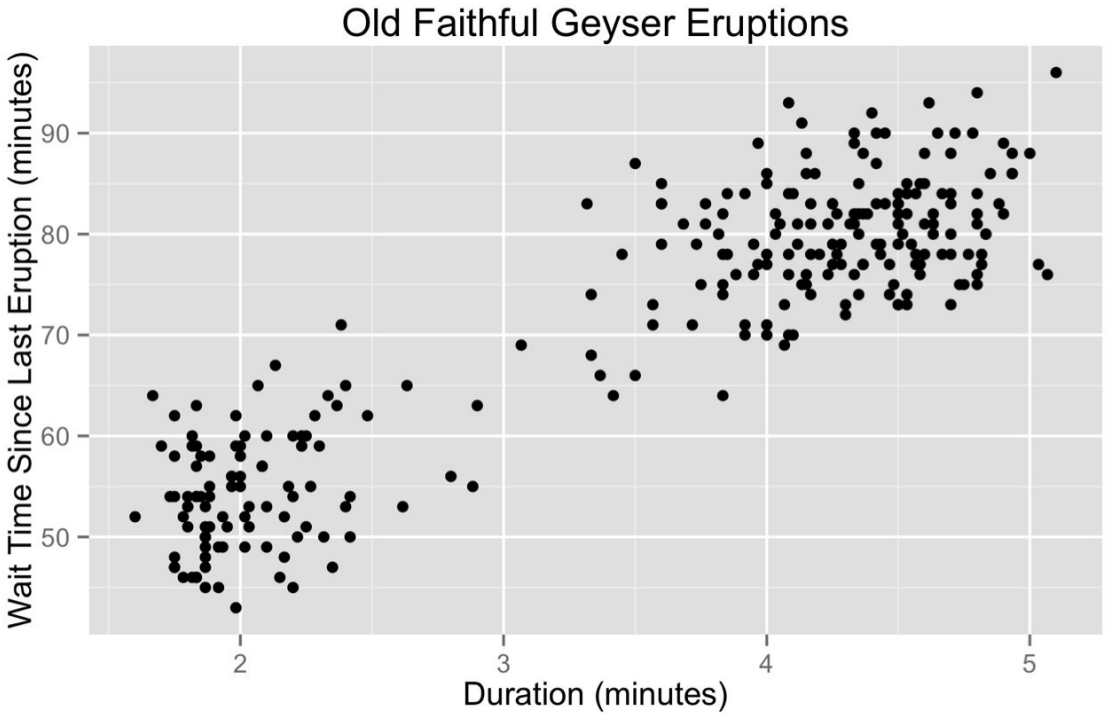


Hierarchical clustering: Pros and Cons

- **Pro**: In some problems, hierarchy of clusters is preferred over flat clusters, such as point linkages
- **Con**: Hierarchical clustering can’t handle big data well, because of its quadratic time complexity $O(n^2)$
- **Con**: Hard to define exact amount of clusters, sensitive to noise and outliers
- **Con**: Lack of interpretability in terms of cluster descriptors
- **Con**: Inability to make corrections after merging/splitting

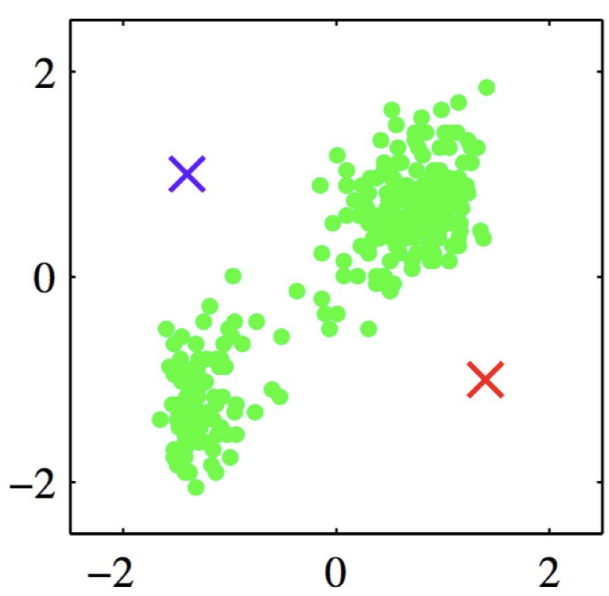
K-means

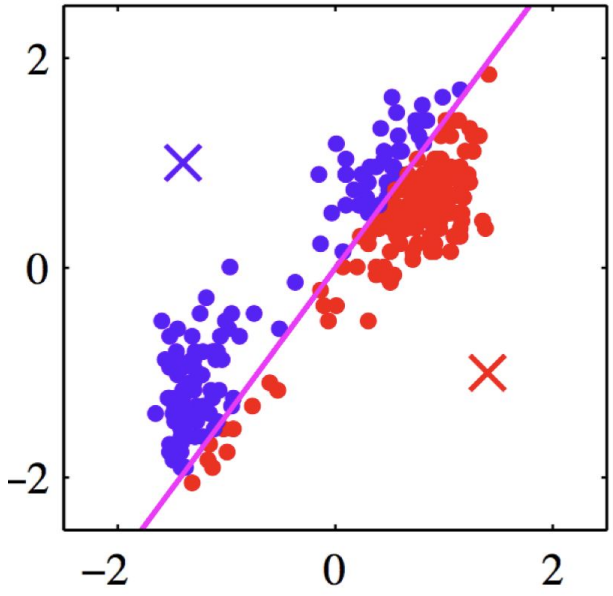
- Looks like two clusters
- Objective: find said clusters algorithmically



K-means: algorithm

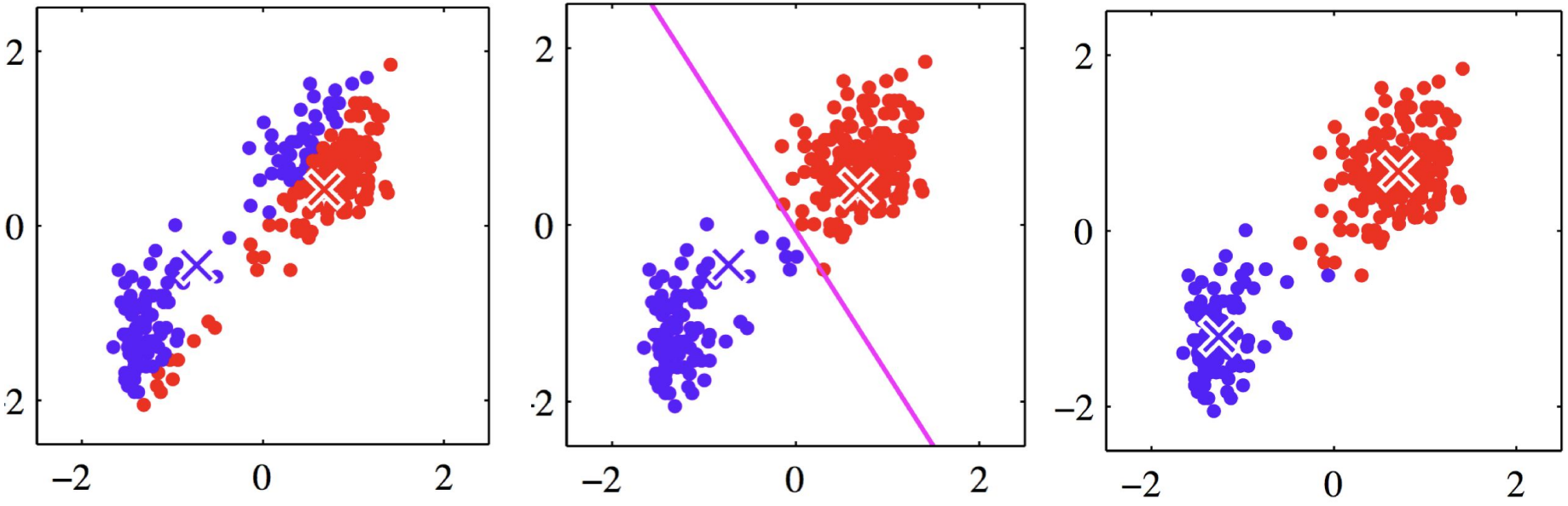
- Standardize the data
- Choose 2 cluster centers(k=2), random points or from the data
- Assign each point to the closest center



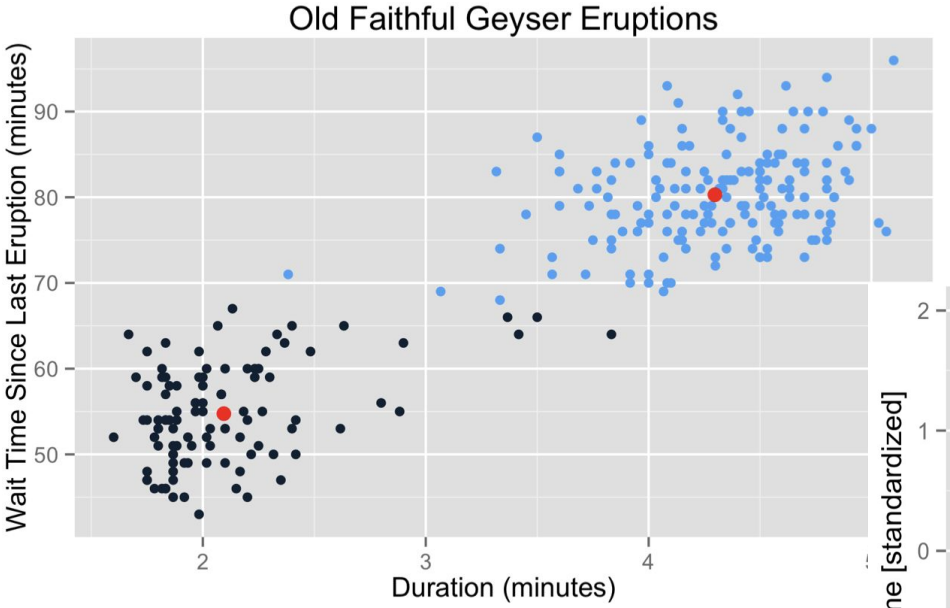


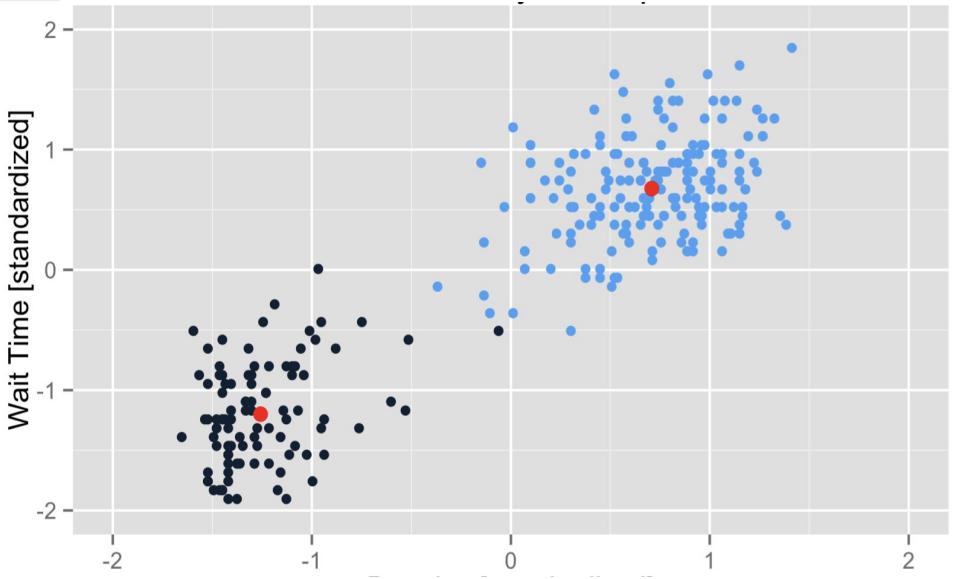
K-means: algorithm

- Compute new cluster centers
- Assign points to the closest center
- Iterate until convergence



K-means: why standardize?





K-means Formalized

Let $\mathcal{X}$ be a space with some distance metric d.

- Most commonly, $\mathcal{X} = \mathbf{R}^d$ and $d(x, x') = \|x - x'\|$.

Dataset $\mathcal{D} = \{x_1, \ldots, x_n\} \subset \mathcal{X}$.

Goal: Partition data $\mathcal{D}$ into k disjoint sets $C_1, \ldots, C_k$.

The **centroid** of C_i is defined to be

$$\mu_i = \mu(C_i) = \arg\min_{\mu \in \mathcal{X}} \sum_{x \in C_i} d(x, \mu)^2.$$

Note: For Euclidean distance on $\mathbf{R}^d$, $\mu(C_i)$ is the mean of C_i.

K-means Formalized

The k-means objective is

$$J_{k\text{-means}}(C_1, \ldots, C_k) = \sum_{i=1}^{k} \sum_{x \in C_i} d(x, \mu(C_i))^2$$

$$= \min_{\mu_1, \ldots, \mu_k \in \mathcal{X}} \sum_{i=1}^{k} \sum_{x \in C_i} d(x, \mu_i)^2$$

input: $\mathcal{D} = \{x_1, \ldots, x_d\} \subset \mathcal{X}$

initialize: Randomly choose initial centroids $\mu_1, \ldots, \mu_k \subset \mathcal{D}$.

repeat until convergence (i.e. until the centroids or clusters repeat):

- $\forall i$, let $C_i = \left\{x \in \mathcal{D} : i = \arg\min_j d(x, \mu_j)\right\}$. (break ties in some arbitrary manner)
- $\forall i$, let $\mu_i = \arg\min_{\mu \in \mathcal{X}} \sum_{x \in C_i} d(x, \mu)^2$. (For Euclidean distance, $\mu_i = \frac{1}{|C_i|} \sum_{x \in C_i} x$)

K-means Formalized

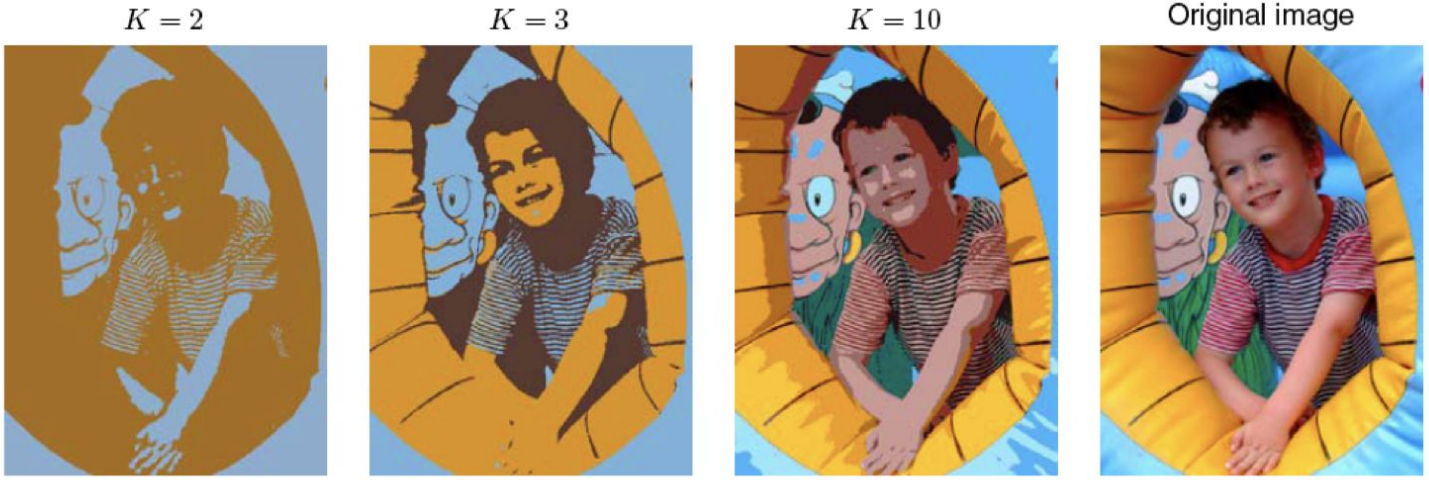
In **vector quantization**, we represent each $x \in C_i$ by the centroid μ_i.

We can think of this as lossy data compression,

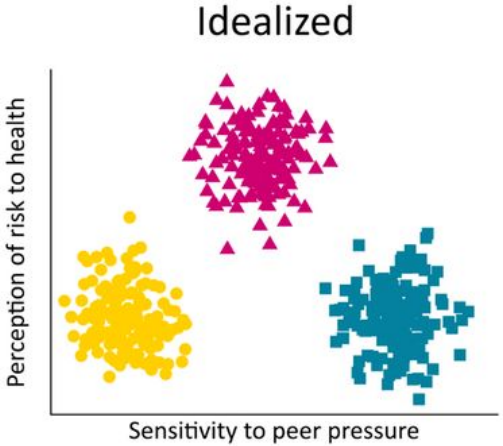
- the k-means objective can be viewed as the reconstruction error.

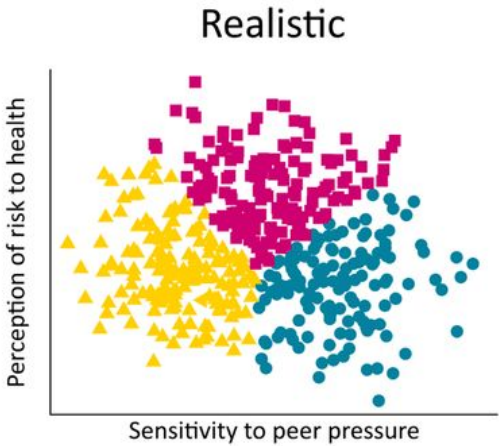
How many bits does it take to represent each point with vector quantization?

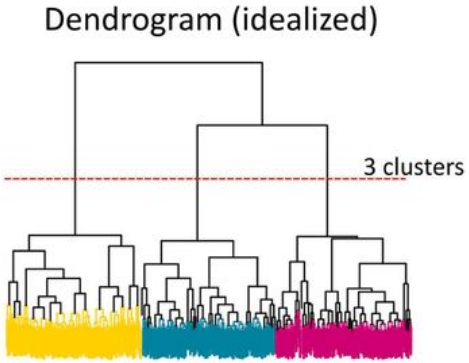
- If $k = 2^d$, then d bits. (Fewer on average if the clusters have unequal sizes.)

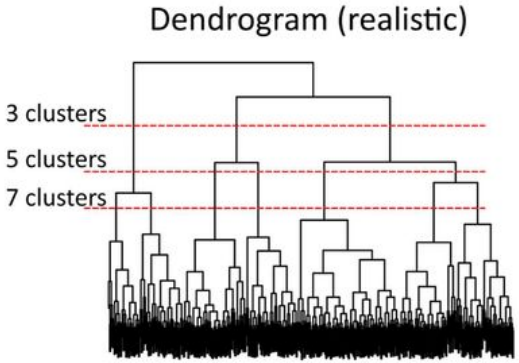


K-means vs Hierarchical



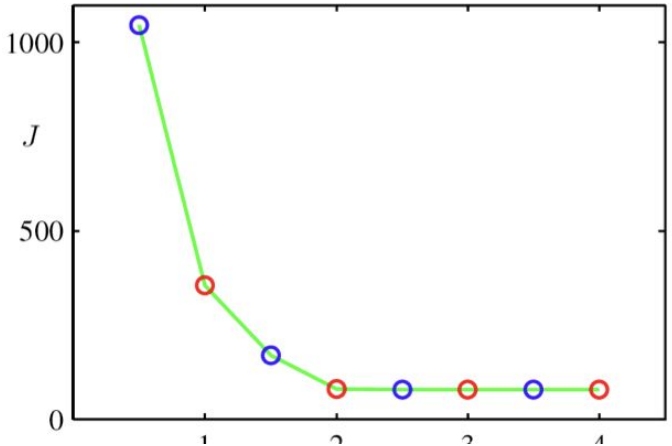






K-means convergence

- K-means algorithm reduces the cost at each iteration.
  - Whenever an assignment is changed, the sum squared distances J of data points from their assigned cluster centers is reduced.
  - Whenever a cluster center is moved, J is reduced.
- Test for convergence: If the assignments do not change in the assignment step, we have converged (to at least a local minimum).
- This will always happen after a finite number of iterations.

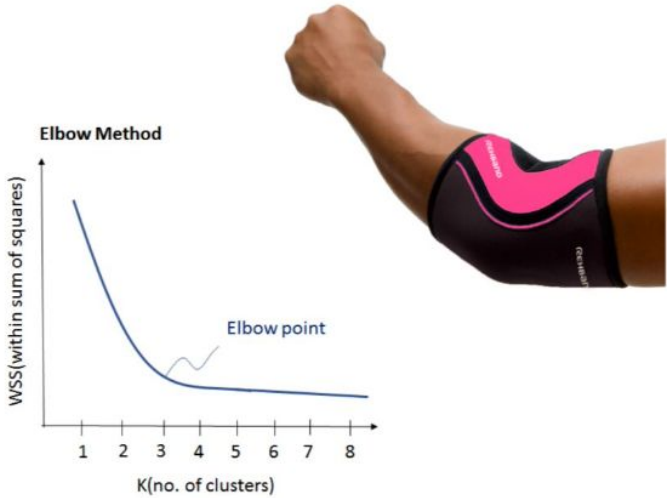


K-means: Choosing the k

- **Elbow method**: increase K until it does not help to describe data better
- We are interested in finding K such that the sum of within-group Euclidean distances is smaller

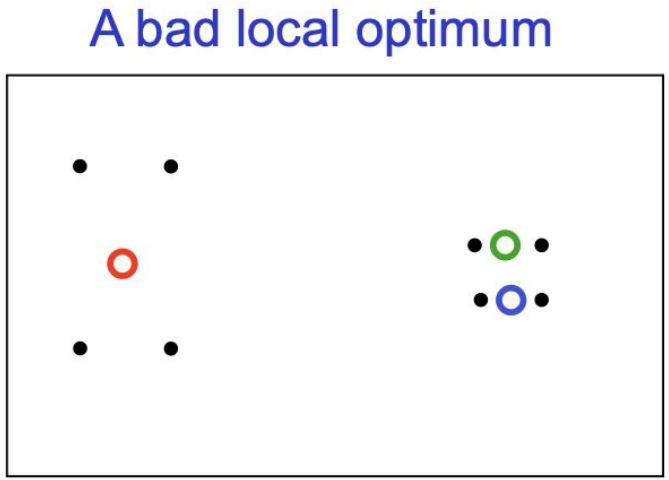
$$J = \sum_{j=1}^{K} \sum_{i=1}^{n_j} \|\mathbf{x}_i^{(j)} - \mathbf{c}_j\|^2,$$

where c_j is the centroid (mean) of the jth cluster



K-means: Local minima

- The objective J is non-convex (so coordinate descent on J is not guaranteed to converge to the global minimum)
- There is nothing to prevent k-means getting stuck at local minima.
- Possible solution is initializing the K-means at random points and running multiple iterations of it, in search of optimal solution.



Initialization by K-means++

- Choose one center uniformly at random among the data points.
- For each data point x not chosen yet, compute D(x), the distance between x and the nearest center that has already been chosen.
- Choose one new data point at random as a new center, using a weighted probability distribution where a point x is chosen with probability proportional to $D^2(x)$.
- Repeat Steps 2 and 3 until k centers have been chosen.
- Now that the initial centers have been chosen, proceed using standard k-means clustering.

K-means: Pros and Cons

- **Pro**: Simple and easy to understand
- **Pro**: Fast and efficient
- **Pro**: Guaranteed convergence!
- **Pro**: Scalable and can handle large datasets efficiently
- **Pro**: Comparatively fast
- **Con**: Requires predefined amount of clusters
- **Con**: Sensitive to initial centroid placement
- **Con**: Assumes spherical clusters
- **Con**: Doesn’t work on categorical data
- **Con**: Is sensitive to outliers

K-medoids

- The K-means algorithm is appropriate when the dissimilarity measure is taken to be squared Euclidean distance
- Using squared Euclidean distance places the highest influence on the largest distances.
- This causes the procedure to lack robustness against outliers that produce very large distances.
- These restrictions can be removed at the expense of computation.

K-medoids

- The only part of the K-means algorithm that assumes squared Euclidean distance is the minimization step of finding the centers (means)
- The algorithm can be generalized for use with arbitrarily defined dissimilarities
- By replacing this step by an explicit optimization with respect to the centers
- In the most common form, centers for each cluster are restricted to be one of the observations assigned to the cluster

K-medoids: Algorithm

1. Choose K, the number of potential clusters
2. Initialise cluster medoids (central points) randomly within the data
3. Instances are clustered to the nearest cluster medoid according to a predefined dissimilarity measure
4. Medoids of each of the K clusters are updated, taking the ones that are closer to all other points in the cluster
5. Steps 3 and 4 are repeated until convergence!

Distance (dissimilarity) measures

- A key component in cluster analysis is the notion of similarity (or dissimilarity) between the individual data objects being clustered
- How can we measure distances (dissimilarities)?
- Choosing a good distance measure is critically important for clustering
- Distance measure formalizes how to describe which objects are “close” to each other, and which are not
- On the same dataset the clustering result can be very different if changing the distance measure

Soft K-means: Algorithm

- Instead of making hard assignments of data points to clusters, we can make soft assignments.
- Initialization: Set K means $\{m_k\}$ to random values
- Repeat until convergence (measured by how much J changes):
  - Assignment: Each data point n given soft “degree of assignment” to each cluster mean k, based on responsibilities

$$r_k^{(n)} = \frac{\exp[-\beta \|\mathbf{m}_k - \mathbf{x}^{(n)}\|^2]}{\sum_j \exp[-\beta \|\mathbf{m}_j - \mathbf{x}^{(n)}\|^2]} \implies \mathbf{r}^{(n)} = \text{softmax}(-\beta \{\|\mathbf{m}_k - \mathbf{x}^{(n)}\|^2\}_{k=1}^K)$$

  - Refitting: Model parameters, means, are adjusted to match sample means of data points they are responsible for:

$$\mathbf{m}_k = \frac{\sum_n r_k^{(n)} \mathbf{x}^{(n)}}{\sum_n r_k^{(n)}}$$

Soft K-means: Pros and Cons

- **Pro**: Overlapping clusters
- **Pro**: Fast and efficient
- **Pro**: Guaranteed convergence!
- **Pro**: Comparatively fast
- **Con**: How to set β?
- **Con**: What to do with clusters with unequal weight and width?

Evaluation

- Unlike supervised learning, clustering often lacks a definitive ”ground truth.”

- How do we mathematically prove that a cluster is ”good”?

- We divide evaluation into two paradigms:
  1. **Internal Evaluation**: We have no labels. We must evaluate the structural integrity of the clusters using only the data itself (distances, densities, variance).
  2. **External Evaluation**: We have benchmarking labels. We evaluate how well our clustering assignments map to the true, known classes.

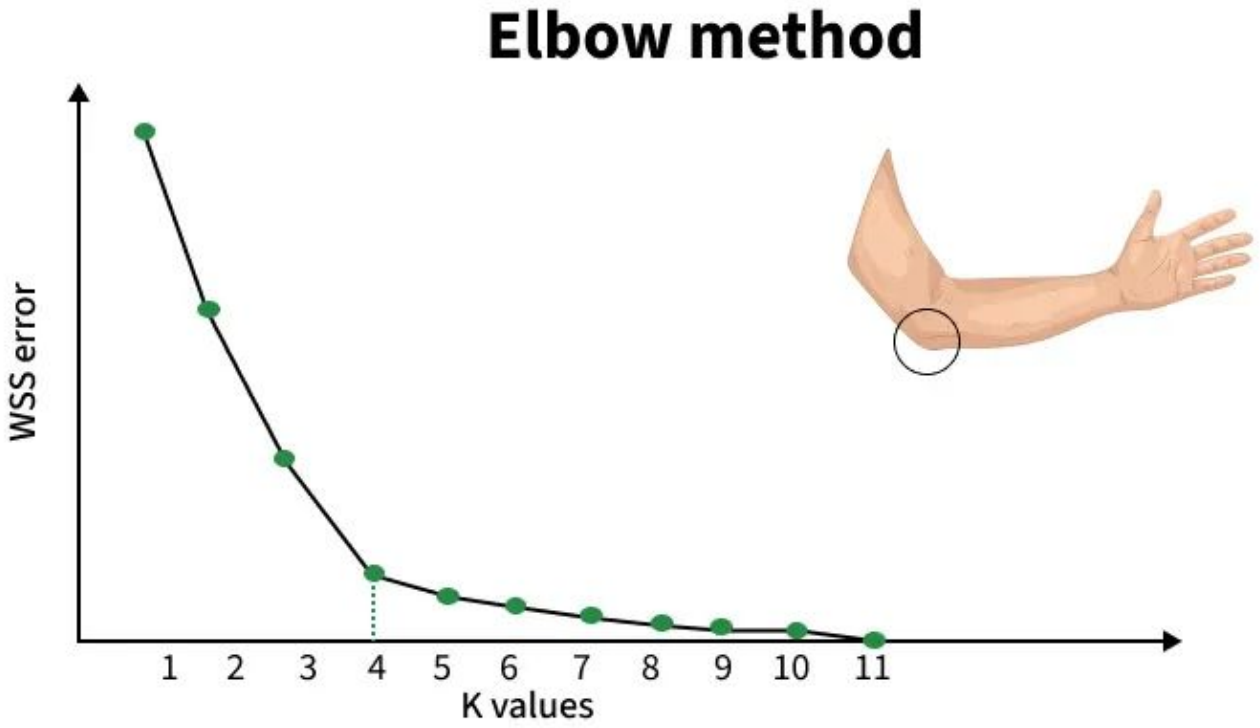
Internal: Inertia

- The fundamental objective function that k-Means actively tries to minimize is Inertia (or Within-Class Sum of Squares (WCSS)).
- It measures how tightly packed the clusters are.

$$WCSS = \sum_{j=1}^{k} \sum_{x_i \in C_j} ||x_i - \mu_j||^2$$

- The Catch: We cannot use WCSS to compare models with different k. As $k \rightarrow N$, WCSS $\rightarrow 0$. A WCSS of 0 is mathematically perfect, but practically useless.
- The Solution: The Elbow Method. We plot WCSS against k and look for the inflection point.

Internal: Inertia



Internal: The Silhouette Coefficient

- Silhouette evaluates both cohesion (how close a point is to its own cluster) and separation (how far it is from other clusters).
- Let a(o) be the mean intra-cluster distance, and b(o) be the mean nearest-cluster distance:

$$a(\boldsymbol{o}) = \frac{\sum_{\boldsymbol{o}' \in C_i, \boldsymbol{o} \neq \boldsymbol{o}'} dist(\boldsymbol{o}, \boldsymbol{o}')}{|C_i| - 1} \qquad b(\boldsymbol{o}) = \min_{C_j : 1 \leq j \leq k, j \neq i} \left\{ \frac{\sum_{\boldsymbol{o}' \in C_j} dist(\boldsymbol{o}, \boldsymbol{o}')}{|C_j|} \right\}.$$

- The **silhouette coefficient** is defined as

$$\mathrm{s(i)} = \frac{\mathrm{b(i)} - \mathrm{a(i)}}{\max(\mathrm{b(i)}, \mathrm{a(i)})}$$

- **Interpretation** ($-1 \leq s(i) \leq 1$):
  - $s(i) \approx 1$: Perfectly clustered. Far from boundaries.
  - $s(i) \approx 0$: Point is exactly on the decision boundary.
  - $s(i) < 0$: Point was likely assigned to the wrong cluster

Internal: Calinski-Harabasz Index

- Also known as the Variance Ratio Criterion. It uses the exact same trace logic from Gaussian Discriminant Analysis!
- It compares the sum of between-clusters dispersion to inter-cluster dispersion.

$$CH = \left[\frac{Tr(B_k)}{Tr(W_k)}\right] \times \left[\frac{N-k}{k-1}\right]$$

- Why use it? It is computationally much faster than Silhouette for massive datasets.
- **Interpretation**: A higher CH score indicates better defined, dense, and well-separated clusters.

Internal: Davies-Bouldin Index

- This metric calculates the worst-case separation-to-density ratio.
- For each cluster C_i, we find its most similar neighboring cluster C_j by looking at the ratio of their internal scatters (s_i, s_j) to the distance between their centroids (d_{ij}).

$$DB = \frac{1}{k}\sum_{i=1}^{k}\max_{j\neq i}\left(\frac{s_i + s_j}{d(c_i, c_j)}\right)$$

- **Interpretation**: Unlike Silhouette and CH, a lower DB index is better. It guarantees that the worst-case cluster overlap is minimized.

External: Adjusted Rand Index (ARI)

- If we happen to know the true ground-truth labels, how do we grade the clusters? We cannot use ”Accuracy” because cluster IDs are arbitrary (Cluster 0 does not inherently mean ”Class A”).
- The Rand Index looks at combinatorics: It counts every pair of samples. If a pair shares the same true label AND falls in the same cluster, that is a success.
- The Adjustment: Standard RI doesn’t account for random chance. The Adjusted Rand Index (ARI) scales the score.
- **Interpretation**:
  - ARI = 1.0: Perfect match between clusters and true labels.
  - ARI ≈ 0.0: The clustering is no better than random guessing.
  - ARI < 0.0: The clustering is worse than random chance.

External: Normalized Mutual Information (NMI)

- A metric borrowed directly from Information Theory.
- **Mutual Information** quantifies the ”amount of information” you obtain about the true classes Y by observing the cluster assignments C.

$$NMI(Y, C) = \frac{2 \cdot I(Y; C)}{H(Y) + H(C)}$$

- I(Y; C) is the mutual information, and H is the entropy (which we remember from Decision Trees!).
- **Interpretation**: Ranges from 0 (no mutual information) to 1 (perfect correlation). An NMI of 1.0 means knowing the cluster assignment gives you 100% of the information needed to deduce the true label.